Inspiration

I'm Dileep, an F-1 student at RIT. Last semester, a friend of mine nearly lost their work authorization in the US. They miscounted the 90-day pre-graduation OPT filing window by two weeks. Two weeks. That's all it takes to go from "on track for a career in the US" to "pack your bags."

There are over 1.1 million international students in the US right now, and every single one of us navigates the same broken process. The rules that determine whether you can live and work here are scattered across USCIS.gov, buried in Reddit threads, paywalled behind $300/hour immigration lawyers, and summarized incorrectly on university websites. One missed deadline, one wrong form, one miscounted day and your legal status is gone.

I've lived this stress firsthand. I built VisaPath because no student should have to lose their future over a confusing government website.

What it does

VisaPath is a personalized AI immigration timeline planner. You answer a few questions about your visa, program, and career goals, and it builds your complete immigration roadmap from today through green card eligibility.

Try it now: visapath-app.azurewebsites.net (use the demo login button to skip registration)

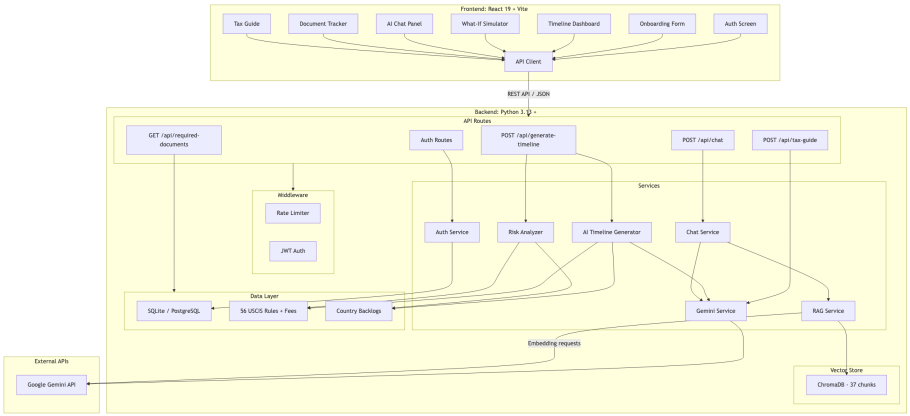

Figure 1: VisaPath System Overview

Timeline Dashboard. The core feature. A vertical, interactive timeline where every deadline is color-coded by urgency: critical (red), high (amber), medium (teal), low (slate). Cards expand to reveal detailed descriptions and step-by-step action items. Past events dim automatically. An "Up Next" badge highlights your most urgent deadline.

Risk Engine. A deterministic rule engine that runs independently from the AI. It flags things that blindside students: 12+ months of CPT killing OPT eligibility, India/China green card backlogs exceeding 10-30 years, unemployment day tracking approaching limits, and the new FY2027 wage-level weighted H-1B lottery that changes your odds based on salary level. No AI dependency means risk alerts are instant and always consistent.

AI Chat. Ask any immigration question and get an answer grounded in official USCIS documentation through RAG, not hallucinated from training data. Sources are cited with every response.

What-If Simulator. Change one variable (graduation date, STEM status, country of origin) and instantly see how your entire timeline shifts. No AI credits consumed.

Tax Guide. Personalized to your visa type, country, treaty benefits, and residency status. Most students don't know their country has a US tax treaty they can claim.

Document Tracker. Step-by-step checklists for OPT, STEM OPT, H-1B, and Green Card filings so you never scramble for paperwork at the last minute.

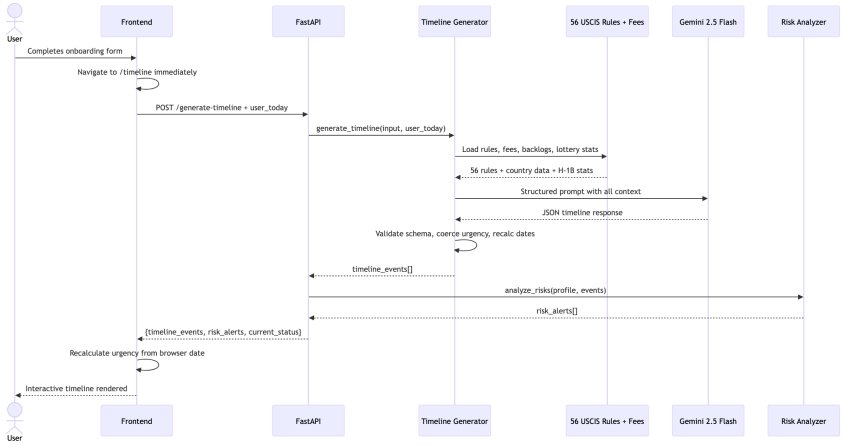

Figure 2: The AI Timeline Generation Engine

How we built it

I built this solo over the hackathon period. The app has two halves: a FastAPI backend (Python 3.13) that handles AI generation, authentication, and data, and a React 19 frontend (TypeScript, Vite 7, Tailwind v4) that renders the interactive timeline.

The part I'm most proud of technically: the AI doesn't get the last word. I don't just throw user data at Gemini and hope for the best. The back-

end injects 56 hardcoded USCIS rules (OPT windows, STEM OPT extensions, cap-gap, unemployment limits), current filing fees, H-1B lottery statistics from FY2024 through FY2027, and country-specific green card backlog data directly into the prompt. The AI generates grounded in these verified rules, not its training data. If Gemini 2.5 Flash hits its rate limit, the system falls back to Gemini 2.0 Flash automatically so the user never sees a failure.

Every generated timeline then passes through a schema validator that rejects responses with missing fields, coerces invalid urgency levels, verifies at least 3 timeline events exist, and recalculates dates from the user's actual local date. If validation fails, it retries up to 2 times. The AI proposes; the validator disposes.

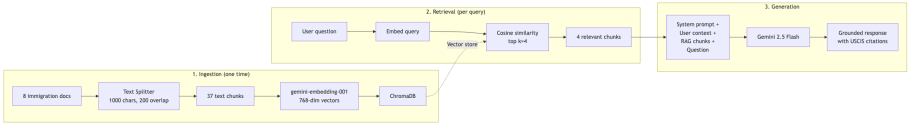

Figure 3: RAG Pipeline - Grounded AI Chat

RAG Pipeline for the chat feature. 8 immigration knowledge base documents (OPT, STEM OPT, CPT, H-1B, green card, F-1 rules, wage-level selection, tax filing) split into 37 chunks, embedded via Google's text embedding model, stored in ChromaDB. Every chat query retrieves the 4 most relevant chunks by similarity and feeds them alongside the student's visa context into the AI. Answers cite real USCIS rules instead of making things up.

Stale-proof dates. The AI generates a timeline once, but a student might not check it for days. Urgency levels would go stale. The frontend recalculates urgency from the browser's local date on every render, so the timeline is always accurate no matter when it was generated.

Auth and rate limiting. JWT with bcrypt hashing, per-IP throttling on auth endpoints, and a per-user credit system with atomic SQL locking to prevent race conditions on concurrent AI requests. An admin dashboard lets me override credits and manage accounts.

Deployment. Azure App Service (free tier, Azure for Students credits) with GitHub Actions CI/CD. Frontend builds to static files served by the backend. ChromaDB runs embedded, no separate database to manage. A demo login button on the login page lets judges skip registration and explore a pre-configured profile instantly. **Note:** the free student tier cold-starts the server after inactivity, so the first load can take up to 5 minutes while Azure spins the instance back up. Subsequent requests are fast.

Challenges we ran into

I underestimated how hard it is to build AI for a domain where mistakes have real consequences. Early versions of the timeline generator

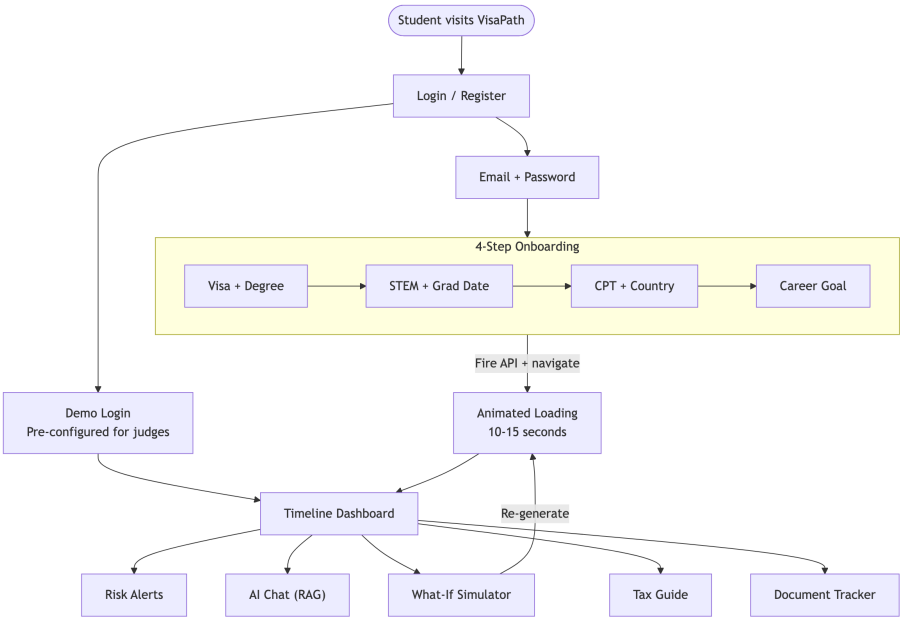

Figure 4: User Flow

would occasionally hallucinate deadlines or mix up filing windows between visa types. For a chatbot that writes poems, that's fine. For a tool where wrong dates could cost someone their legal status, that's unacceptable. I ended up hardcoding 56 USCIS rules and injecting them into every prompt as ground truth, then adding a full validation pipeline on top. It tripled the backend complexity, but the output went from "probably right" to "right enough to trust."

The 15-second AI generation nearly killed the user experience. My first version had users staring at a spinning button on the onboarding form for 10-15 seconds while Gemini processed. I watched a friend try it and they hit the back button after 8 seconds, thinking it was broken. I had to completely rethink the flow: now the form fires the API call and immediately navigates to the timeline page, which shows an animated loading experience with progress steps and rotating "Did you know?" immigration facts. Same wait time, completely different perception.

Timezone bugs caused wrong urgency colors for almost a full day before I caught it. The server runs UTC on Azure, but a student in California at 11pm on February 16th should see February 16th, not the 17th. I only noticed because I was testing late at night and a deadline that should have been "critical" showed as "high." The fix: the frontend sends its local date with every request, and the backend uses that instead of server time for all date calculations. Simple

in hindsight, but it was buried under other issues for hours.

I scoped way too aggressively at the start. My original plan had email notifications, calendar export, PDF reports, and a community forum. By day 2, I realized I wouldn't finish the core timeline feature if I kept building sideways. I cut everything except the 6 features that shipped. The document tracker checkbox state still doesn't persist across page refreshes because I ran out of time. It bothers me, but it was the right tradeoff.

Accomplishments that we're proud of

I shared VisaPath with 15 classmates at RIT, all international students on F-1 visas. The most useful feedback:

One student discovered she only had 23 days left in her OPT filing window. She was planning to file next month, which would have been too late. Another used the what-if simulator to compare staying on OPT vs applying through a cap-exempt employer, and found a path that skips the H-1B lottery entirely through a university-affiliated research position. A third had no idea India has a US tax treaty for students that most people don't claim. Neither did his friends.

Beyond the user feedback: the timeline produces accurate, differentiated results across real scenarios. An F-1 STEM student from India gets a completely different roadmap than a non-STEM student from Brazil or an H-1B holder filing for a green card. Each one generates the right deadlines, the right risks, and the right action items for their specific situation.

The system handles edge cases I didn't even plan for. A student who's used 11 months of CPT sees a critical warning about losing OPT eligibility. Someone from India gets a frank risk alert about 10-30+ year green card wait times with alternate path suggestions. An entry-level hire sees how the FY2027 wage-level lottery changes their H-1B odds.

What we learned

Building AI for high-stakes domains is a completely different discipline than building a chatbot. You can't prompt and hope. The 56 hardcoded rules, 37-chunk RAG grounding, model fallback chain, and output validation with retry logic are all layers of guardrails. Each one exists because I caught the AI getting something wrong during testing. "Good enough" isn't good enough when someone's legal status is on the line.

UX around AI latency matters as much as the AI output itself. I spent more time on the 15-second loading experience than on several actual features. Nobody waits 15 seconds for a spinner. Everyone waits 15 seconds for a beautiful loading animation that teaches them something while they wait. The perception of speed is a design problem, not a performance problem.

Solo hackathons teach you scope discipline the hard way. I cut 4 planned features (email alerts, calendar export, PDF reports, community forum) to ship the 6 that mattered. Every one of those cuts was painful in the moment and obviously correct in hindsight. The features that shipped are solid because they got all the time that the cut features would have eaten.

What's next for VisaPath

Persist document tracker state. The checkbox state resets on page refresh right now. It's the first thing I'm fixing.

Deadline notifications. Email and browser push alerts before critical dates. This is the most requested feature from the students who tested it.

More visa types. J-1, L-1, and O-1 with dedicated timeline logic. The architecture supports it, the knowledge base documents just need to be written.

Live USCIS processing times. Right now the processing time estimates are hardcoded from my last research pass. Pulling real-time data from the USCIS API would keep timelines accurate without manual updates.

Cap-exempt employer database. A lookup tool showing which companies and institutions let you skip the H-1B lottery entirely. This came directly from user feedback during testing.

No student should lose their future because of a confusing government website. VisaPath makes sure they don't.